TEMA 1

Desarrollo de software

Del problema al producto: ciclo de vida, modelos y metodologías

Propósito del tema

Comprender el proceso, utilizar las herramientas con criterio y documentar decisiones reproducibles.

ENTORNOS DE DESARROLLO · 1.º DAM

Qué aprenderás

- Distinguir software de sistema, de aplicación y de desarrollo.
- Explicar cómo se transforma el código fuente en un programa ejecutable.
- Reconocer las fases, productos y responsables del ciclo de vida.
- Comparar modelos predictivos, iterativos y metodologías ágiles.

> **Ruta de aprendizaje**
> Conceptos → decisiones técnicas → ejemplos en Java → comprobación y documentación.

Índice del tema

1. Software: definición y clasificación
2. Relación entre hardware y software
3. Lenguajes, código y traducción
4. Herramientas y entorno de desarrollo
5. Ciclo de vida del software
6. Requisitos, diseño y construcción
7. Pruebas, despliegue y mantenimiento
8. Modelos predictivos del ciclo de vida
9. Modelos iterativos y evolutivos
10. Desarrollo ágil
11. XP y Scrum
12. Elegir y adaptar el proceso

1. Software: definición y clasificación

El software es el conjunto de programas, datos, reglas y documentación que permite a un sistema informático realizar tareas. No es únicamente código: también incluye configuración, pruebas, manuales y decisiones de diseño.

Tipo	Finalidad	Ejemplos
Software de sistema	Gestiona el hardware y ofrece servicios básicos	Sistema operativo, controladores, utilidades
Software de aplicación	Resuelve necesidades de las personas usuarias	ERP, navegador, videojuego, app móvil
Software de desarrollo	Ayuda a construir y mantener otros programas	JDK, Eclipse, Git, depurador

> **Idea clave**
>
> Un producto software de calidad combina funcionalidad, fiabilidad, facilidad de uso, eficiencia, mantenibilidad y portabilidad.

2. Relación entre hardware y software

El hardware ejecuta instrucciones y almacena datos; el software decide qué operaciones deben realizarse y en qué orden. Entre una aplicación y el procesador intervienen varias capas que aportan abstracción y portabilidad.

```
Persona usuaria
      ↓
Aplicación
      ↓
Bibliotecas y máquina virtual
      ↓
Sistema operativo
      ↓
Hardware
```

- El sistema operativo administra memoria, procesos, archivos y dispositivos.
- Las bibliotecas reutilizan soluciones y ocultan detalles de bajo nivel.
- Una máquina virtual, como la JVM, desacopla el programa de la plataforma física.

3. Lenguajes, código y traducción

Los lenguajes de programación permiten expresar algoritmos. El código fuente es legible por personas; el código objeto es el resultado de la traducción; el ejecutable reúne el código traducido y las bibliotecas necesarias para su ejecución.

Enfoque	Proceso	Resultado
Compilación	Traduce el programa antes de ejecutarlo	Código objeto o ejecutable
Interpretación	Traduce y ejecuta durante la ejecución	No genera necesariamente un ejecutable independiente
Java	javac genera bytecode; la JVM lo verifica y ejecuta	Archivos .class portables entre JVM compatibles

```
Programa.java  --javac-->  Programa.class  --JVM-->  ejecución
```

Compilar no garantiza corrección

La ausencia de errores sintácticos no impide que existan fallos lógicos o incumplimientos de requisitos.

4. Herramientas y entorno de desarrollo

El desarrollo combina herramientas especializadas: editor, compilador, sistema de construcción, depurador, pruebas, control de versiones y documentación. Un IDE integra muchas de ellas en una interfaz común.

Herramienta	Aporta
Editor	Escritura, coloreado, navegación y ayuda de código
Compilador	Traducción y diagnóstico de errores
Depurador	Ejecución controlada y observación del estado
Pruebas	Verificación repetible del comportamiento
Git	Historial, colaboración y recuperación de cambios

Trazabilidad

Una herramienta es valiosa cuando deja evidencia: versión, autoría, resultados de prueba y relación con el requisito.

5. Ciclo de vida del software

El ciclo de vida organiza el trabajo desde la necesidad inicial hasta la retirada del producto. Aunque un modelo muestre fases consecutivas, en proyectos reales existe retroalimentación entre ellas.

Fase	Pregunta principal	Productos habituales
Planificación y análisis	¿Qué problema y restricciones existen?	Alcance, requisitos, estimación, riesgos
Diseño	¿Cómo se organizará la solución?	Arquitectura, modelos, interfaces, datos

Fase	Pregunta principal	Productos habituales
Implementación	¿Cómo se materializa el diseño?	Código fuente, configuración, documentación técnica
Pruebas	¿Cumple y funciona con confianza?	Casos, evidencias, incidencias e informes
Despliegue	¿Cómo llega al entorno real?	Paquete, migración, manuales y formación
Mantenimiento	¿Cómo evoluciona de forma segura?	Correcciones, versiones y registro de cambios

6. Requisitos, diseño y construcción

Requisitos

Los requisitos funcionales describen servicios y reglas de negocio. Los no funcionales expresan cualidades y restricciones: rendimiento, seguridad, accesibilidad, compatibilidad o disponibilidad. Deben ser claros, verificables y trazables.

Diseño e implementación

El diseño distribuye responsabilidades entre componentes y anticipa interfaces, datos y dependencias. La implementación traduce esas decisiones a código, manteniendo convenciones, revisiones y versiones pequeñas que puedan probarse.

> **Criterio de aceptación**
>
> Cada requisito importante debe poder asociarse con una condición observable que permita decidir si está cumplido.

7. Pruebas, despliegue y mantenimiento

Las pruebas buscan información sobre riesgos y defectos antes de la entrega. El despliegue prepara el software, los datos y a las personas para el uso real. Después comienza una etapa de mantenimiento que suele ocupar gran parte de la vida del producto.

Mantenimiento	Objetivo	Ejemplo
Correctivo	Eliminar defectos	Corregir un cálculo erróneo
Adaptativo	Responder a cambios del entorno	Compatibilidad con una nueva versión de Java
Perfectivo	Mejorar o ampliar el producto	Reducir tiempos de respuesta
Preventivo	Reducir problemas futuros	Refactorizar código duplicado y añadir pruebas

8. Modelos predictivos del ciclo de vida

Modelo en cascada

Organiza fases principalmente secuenciales y exige definir pronto el alcance. Facilita la planificación y la documentación, pero responde peor a requisitos cambiantes y retrasa la obtención de software utilizable.

Modelo en V

Relaciona cada nivel de definición y diseño con un nivel de prueba. Refuerza la verificación temprana, aunque conserva una estructura predictiva.

> **Cuándo encaja**
>
> Puede ser adecuado cuando el dominio es estable, existe regulación intensa y los cambios tardíos son especialmente costosos.

9. Modelos iterativos y evolutivos

Modelo	Mecánica	Ventaja	Riesgo
Incremental	Entrega partes funcionales sucesivas	Valor temprano y priorización	Arquitectura fragmentada si no se planifica
Prototipado	Construye una versión para aprender	Aclara requisitos y experiencia de usuario	Confundir prototipo con producto final
Espiral	Itera guiado por análisis de riesgos	Adecuado para proyectos complejos	Mayor coste de gestión y experiencia

Iterar significa repetir actividades para aprender y mejorar. Incrementar significa ampliar el producto con una porción utilizable. Un proyecto puede ser simultáneamente iterativo e incremental.

10. Desarrollo ágil

La agilidad propone ciclos breves, colaboración continua, software funcionando y capacidad de adaptación. No elimina la planificación ni la documentación: busca que sean proporcionales, útiles y actualizadas.

- Entregar valor con frecuencia y recibir retroalimentación real.
- Trabajar con equipos multidisciplinares y responsabilidad compartida.
- Mantener calidad técnica para que el cambio siga siendo sostenible.
- Inspeccionar resultados y adaptar el proceso con regularidad.

> **Error frecuente**
>
> Ágil no significa improvisado. Cada iteración debe tener objetivo, criterios de aceptación, control de calidad y una definición clara de terminado.

11. XP y Scrum

Enfoque	Elementos destacados	Finalidad
XP	Programación por parejas, integración continua, diseño simple, refactorización y pruebas	Excelencia técnica y respuesta segura al cambio
Scrum	Product Backlog, Sprint, revisión, retrospectiva y roles definidos	Gestionar trabajo complejo mediante transparencia, inspección y adaptación

En Scrum, el Product Owner ordena el valor del producto; el Scrum Master favorece el marco y elimina impedimentos; el equipo construye un incremento terminado durante el Sprint.

```
Necesidad → Product Backlog → Sprint Planning → Sprint
                                    ↓
                          Incremento terminado
                                    ↓
                        Review + Retrospective
```

12. Elegir y adaptar el proceso

No existe un modelo universal. La elección depende de estabilidad de requisitos, criticidad, tamaño, experiencia del equipo, disponibilidad del cliente, regulación y coste del cambio.

Contexto	Orientación razonable
Requisitos estables y auditables	Proceso predictivo con documentación y revisiones formales
Producto nuevo con incertidumbre	Iteraciones cortas, prototipos y validación frecuente
Riesgo técnico elevado	Espiral, pruebas de concepto y decisiones basadas en evidencia
Servicio en evolución continua	Ágil, automatización y entrega incremental

> **Pregunta de cierre**
>
> El mejor proceso es el que hace visibles los riesgos, produce evidencia y permite entregar valor con calidad sostenible.

Síntesis y autoevaluación

- El software incluye código, datos, configuración y documentación.
- El ciclo de vida conecta requisitos, diseño, construcción, pruebas, despliegue y mantenimiento.
- Los modelos predictivos priorizan previsibilidad; los iterativos priorizan aprendizaje y adaptación.
- Scrum organiza el trabajo; XP aporta prácticas técnicas para sostener la calidad.

Reto final · Comprueba que dominas el tema

Compara el desarrollo de una aplicación de gestión académica con cascada, incremental y Scrum. Indica entregables, riesgos, momentos de prueba y criterios para elegir un enfoque.

Material base

Tema reorganizado, ampliado y redactado en español a partir de entornos-desarrollo-de-software.pdf, aportado por el docente. No se reproducen los ejercicios del documento original.